

附表 A-1 常见的 Linux 命令和 Windows 命令

分 类	Linux 命令	Windows 命令
文件列表	ls	dir
改变/创建/删除目录	cd/mkdir/rmdir	cd/md/rd
显示文件内容	cat/more	type/more
比较文件内容	diff	fc
修改文件属性	chmod	attrib
复制文件	cp	copy/xcopy
删除文件	rm	del
文件改名	mv	ren
回显	echo	echo
关闭命令行	exit	exit
在文件中查找字符串	grep	find
查看/修改环境变量	set	set
帮助	man <命令>	help <命令>

A.2 操作系统脚本编程入门

读者如果不学习脚本的编写，就无法让命令行发挥最大威力。编写脚本和编写 C 语言程序有几分相似，但也有一些不同。下面先来看一个常见任务：不停地随机生成测试数据，分别运行两个程序并对比其结果。这个任务被形象地称为“对拍”。

A.2.1 Windows 下的批处理

Windows 下的批处理程序如下：

```
@echo off
:again
r > input           ;生成随机输入
a < input > output.a
b < input > output.b
fc output.a output.b > nul      ;比较文件
if not errorlevel 1 goto again  ;相同时继续循环
```

第 1 行表明接下来的各个命令本身并不会回显。如果不理解，试着把这一行去掉就明白了。第 2 行是一个标号，后面的 goto 语句用得上。接下来调用数据生成器 *r*，把输入数据写到文件 *input* 中，然后分别执行 *a* 和 *b*，得到相应的输出，然后用命令 *fc* 比较它们。注意，*fc* 命令有输出，但我们对此不感兴趣，因此重定向到一个名为 *nul* 的设备中，它就好比一个黑洞。另一个有意思的设备是 *con*，代表标准输入输出。例如，命令 *copy con con* 的含义是

直接把标准输入复制到标准输出（尽管有些傻）。试一试，建立一个只包含一条语句的“C 程序”：`#include<con>`，用命令行编译一下试试——很不幸，看上去编译器“死掉”了，尽管它其实是在读键盘。如果在设计一个基于 Windows 的在线评测系统，小心好事者用它来愚弄你的系统！另一方面，千万不要在正式比赛中使用这个伎俩——它很可能让你失去比赛资格。

最后一行是整个批处理程序的关键——只有当比较文件相同时才执行 `goto`，否则立刻终止程序。这样，就有机会好好研究一下这个 `input` 文件，看看两个程序的输出到底为什么不同。读者也许会问，这个 `if not errorlevel 1` 到底是什么意思呢？它是在测试上一个程序（在本例中，就是 `fc` 程序）的返回码。`if errorlevel num` 的意思是“如果返回码大于或者等于 `num`”，因此 `if not errorlevel 1` 的意思是，“如果返回码小于 1”。事实上，当且仅当文件相同时，`fc` 程序返回 0。如果不确定程序的返回码是多少，可以在程序执行完毕后用 `echo %errorlevel%` 命令输出返回码。

你自己编写的程序的返回码是多少呢？这要看在 `main` 函数的最后 `return` 的是多少。返回码 0 往往代表“正常结束”，因此本书的正文部分才建议用 `return 0`。典型的评分程序将在执行选手程序之后判断它的返回码，如果非 0，则直接认为程序非正常退出，根本不去理会输出是否正确。说到这里，你也许已经想到一种故意让返回码非 0 的情况了——输出检查器。对于答案不唯一的情况（例如，走迷宫时要求输出最短路径，但不必是字典序最小的），对拍时不能简单地用 `fc` 命令比较文本内容，而应该单独编写一个程序，这个程序应当在答案不一致时返回 1，以便上面的批处理程序及时终止。

上面的程序应以 `.bat` 为扩展名保存，并且在执行时也可以省略扩展名。如果同时存在 `abc.bat` 和 `abc.exe`，将执行 `abc.exe`。但如果主文件名和系统命令重名，则连 `exe` 文件也无法执行，如 `path.exe`。

A.2.2 Linux 下的 Bash 脚本

下面是上述程序的 Linux 版：

```
#!/bin/bash
while true; do
    ./r > input                #生成随机数据
    ./a < input > output.a
    ./b < input > output.b
    diff output.a output.b     #文件比较
    if [ $? -ne 0 ] ; then break; fi #判断返回值
done
```

和 Windows 版没有太大的不同，但需要注意的是，Linux 中的设备名和 Windows 有所不同，而且也没有必要执行类似 `@echo off` 的命令——命令本来就不会回显。需要注意的是，如果在 Windows 下编写 Linux 脚本，复制到 Linux 后需要去掉所有的 `\r` 字符，否则解释器会报错。